

I. PROGRAMMATION DYNAMIQUE.

1. EXEMPLE DU RENDU DE MONNAIE.

a) Solution Gloutonne



La méthode « usuelle » pour rendre la monnaie est celle de l'algorithme glouton : tant qu'il reste quelque chose à rendre, choisir la plus grosse pièce (c'est le choix « glouton ») qu'on peut rendre (sans rendre trop).

Ex 1 : si l'on souhaite rendre 8 et si l'on ne possède que des pièces de 1 de 4 et de 5
la méthode gloutonne donne :

Si l'on recherche un rendu monnaie avec le moins de pièces possibles, la méthode gloutonne fournit-elle une solution optimale ?

Rq1. On appelle canonique un système de pièces pour lequel l'algorithme glouton donne une solution optimale quelle que soit la valeur à rendre.
Presque tous les systèmes de pièces existant dans le monde sont canoniques (dont celui de l'euro), ce qui justifie a posteriori la méthode « usuelle » de rendu de monnaie.

Ex 2 : Un distributeur automatique dispose du système de pièces suivants :
2 €, 1 €, 0.50 €, 0.20 €, 0.10 €, 0.05 €, 0.02 €, 0.01 €.
Il applique la méthode gloutonne et doit rendre la somme de 5.33 € avec le moins de pièces possibles.
Il décompose : 5.33 =

Exo 1. Les pièces à disposition sont mémorisées dans un tuple dont les éléments sont les différentes pièces disponibles dans un ordre indifférent (par exemple `euros=(2,1,.5,.2,0.1,0.05,0.02,0.01)`.)
Écrire une fonction `rendre_monnaie(pièces,s)` qui prend en argument `pieces`, le système de pièces utilisé et la somme `s` d'argent que l'on doit décomposer dans le système de pièces, qui renvoie la décomposition sous la forme d'une liste en appliquant l'algorithme glouton.
Vérifier en particulier que `rendre_monnaie(euros,5.33)` renvoie `(2,2,1,0.2,0.1,0.02,0.01)`.
et que `rendre_monnaie((1,4,5),8)` renvoie `(5,1,1,1)` (solution non optimale)



[algo_RenduMonnaie_glouton.py](#)

b) Solution optimale



Pour simplifier les notations, on ne considère que les pièces ou billets de sommes entières (`euros=(1,2,5,10,20,50,100,200)` par ex.) et les sommes à rendre entières également.
De plus on considère que tous les systèmes de pièces contiennent la valeur 1.

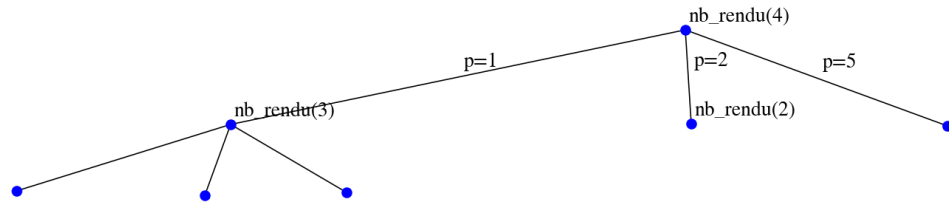


Pour `s` entier, soit `rendu=rendre_monnaie((1,4,5),s)`, un ensemble de pièces réalisant une solution au problème du rendu de monnaie de `n` avec les pièces 1, 4 et 5 (par exemple), alors :
Si `rendu` possède une pièce de valeur `p` alors la même liste `rendu` à laquelle on a retiré la valeur `p` est le retour de `rendre_monnaie((1,4,5),s-p)`.

Exo 2. On donne le code suivant [algo_RenduMonnaie_optimal.py](#):

1. Tester ce code avec les tests fournis. Quelle semble être le rôle de la fonction `nb_rendu()` ?

2. Compléter l'arbre des appels de cette fonction récursive et surligner le chemin de la solution optimale du rendu monnaie.



3. Justifier que cette fonction renvoie le nombre de pièces de la solution optimale du rendu monnaie.
4. Compléter :
- $$\text{nb_rendu}(\text{pieces}, s) = \begin{cases} \dots & \text{si } s=0 \\ \min(\{ \dots & \text{tel que } p \in \text{pieces et } p \leq s \}), \text{ sinon} \end{cases}$$
- ```
print("rendu glouton 29 en euros) : ", glouton.rendre_monnaie(euros, 29))
print("nb-rendu...")
print("nb de pièces optimal pour rendre 29 :", nb_rendu(euros, 29))
```
- D'où peut provenir le problème constaté ?
5. Equiper le code de la fonction **nb\_rendu(pieces, s)** de façon à compter le nombre d'appels récursifs à la fonction et en particulier, par exemple, le nombre d'exécutions de **nb\_rendu(euros, 5)**.



[algo\\_RenduMonnaie\\_optimal\\_dyn.py](#)

La **programmation dynamique** vise à éviter la répétition de calculs déjà exécutés en mémorisant leurs résultats.

## 2. PROGRAMMATION DYNAMIQUE



La programmation dynamique et/ou la mémorisation sont deux techniques très proches qui s'appuient sur l'idée naturelle suivante : ne pas recalculer deux fois la même chose.

- a) **Méthode de mémorisation** (terme issu de l'anglais *memoization*, lui-même issu de *memo* (pense-bête en français)).

A partir de la programmation récursive, l'intégration d'un dictionnaire (ou parfois d'un tableau) qui enregistre les résultats déjà calculés, et sa consultation avant de déclencher un nouveau calcul permet une mise en œuvre simple de la mémorisation.

### Exo 3. Implémenter la fonction **nb\_rendu\_memo()**

```
def nb_rendu_memo(pieces, s, dico={}):
 """prend en paramètre le systeme de pieces (tuple) et s, la somme à rendre,
 et dico le dictionnaire des calculs déjà effectués
 renvoie la taille de la liste optimale des pièces rendues"""
```

**Exo 4.** A l'aide du code de mesure de performances ci-dessous, constater l'évolution des performances pour un rendu de 29 en système euros.

```
import timeit, functools
from time import perf_counter

def temps_d_execution(f,x,y):
 """mesure le temps d'exécution de f(x,y)"""
 print("{} avec timeit et x={} et y={}".format(f,x,y))
 print(timeit.timeit(functools.partial(f,x,y),number=1))
```

**Rq2** La mémorisation est une technique descendante (on commence à résoudre le problème donné en le décomposant en problème de taille inférieure de manière récursive). On ne calcule cependant que des problèmes plus petits mais pas forcément tous :  
**nombre\_rendu\_memo((2,3),5)** ne va nécessiter que le calcul de **nombre\_rendu\_memo((2,3),3)** ou de **nombre\_rendu\_memo((2,3),2)** pour conclure. Il n'aura pas besoin de **nombre\_rendu\_memo((2,3),4)**

**b) Deuxième méthode : construction itérative par tabulation**

**Rq3** Le deuxième type de programmation dynamique est une technique ascendante (on commence à résoudre tous les problèmes de taille inférieure à partir du sous-problème trivial, en visant le problème donné)

la programmation est alors itérative ; l'intégration d'un tableau qui enregistre les résultats calculés progressivement, et sa consultation avant de déclencher un nouveau calcul permet une mise en œuvre simple de la programmation dynamique.

**Exo 5.** On veut implémenter la fonction **nb\_rendu\_dyn()**, dans une version itérative qui utilisera  
 la propriété déjà écrite : **nb\_rendu(pieces,s)=min({1+nb\_rendu(pieces,s-p) tel que p ∈ pieces et p ≤ s})**,  
 et un tableau **tab** dans lequel on **enregistre** progressivement les tailles des solutions optimales afin de ne pas recalculer un résultat déjà connu.

```
def nb_rendu_dyn(pieces,s):
 """prend en paramètre le système de pièces (tuple) et s, la somme à rendre,
 et renvoie la taille de la liste optimale des pièces rendues"""
```

**a.** Quel sera le tableau **tab** construit par cette fonction quand on exécute **nb\_rendu\_dyn((1,6,10),8)** ?

| s            | 0  | 1   | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|--------------|----|-----|---|---|---|---|---|---|---|
| rendu_opt    | [] | [1] |   |   |   |   |   |   |   |
| nb_rendu_opt | 0  | 1   |   |   |   |   |   |   |   |

**tab=**

**b.** Implémenter et tester cette fonction.

**Rq4.** L'appel récursif de la version memo est remplacé par un accès au tableau **tab**.

**Exo 6.** Le programme précédent donne le nombre minimal de pièces nécessaires au rendu monnaie mais oublie de renvoyer la monnaie (ou plutôt la liste des pièces à rendre).

Coder une fonction **rendre\_monnaie\_dyn()**, une deuxième version de **rendre\_monnaie()**, qui renvoie la monnaie, en intégrant au modèle de **nombre\_rendu\_dyn()**, un deuxième tableau qui contient pour chaque somme entre 0 et  $s$ , une solution optimale (liste des pièces à rendre) pour cette somme.

**Rq5.** De manière générale, le coût en temps est proportionnel à  
Le coût en espace est lui proportionnel à  $s$  (pour le tableau **tab**) (potentiellement plus coûteux que l'algorithme glouton, mais donne la solution optimale pour tous les système monétaires)

## II. EXOS DYNAMIQUES

**Exo 7.** La célèbre suite de Fibonacci est définie par  $u_0 = u_1 = 1$  et la relation de récurrence  $u_{n+2} = u_{n+1} + u_n$ .

1. Coder la fonction récursive **fibo(n)** qui renvoie  $u_n$ .
2. Coder la fonction « mémoïsée » **fibo\_memo(n)** qui renvoie  $u_n$ .
3. Coder la fonction **fibo\_dyn(n)** qui renvoie  $u_n$  en itératif dynamique.
4. Mesurer les performances en temps de chaque version (utiliser **timeit**)

**Exo 8.** En UTF-8 les caractères sont stockés sur 1 à 4 octets, on se pose la question du nombre de combinaisons de caractères que l'on peut stocker sur  $n$  octets (c'est plus un problème de dénombrement que d'optimisation).

Par exemple pour  $n = 4$ , les possibilités de stockage sont :

- (1,1,1,1) : 4 caractères de 1 octet
- (1,1,2) : 2 caractères de 1 octet plus un de deux.
- (1,2,1)
- (2,1,1)
- (2,2)
- (1,3)
- (3,1)
- (4)

1. Trouver toutes les combinaisons de taille de caractères tenant dans 5 octets.
2. Si on note  $u_n$  ce nombre, établir une relation de récurrence sur  $u_n$  avec  $n \geq 4$ .
3. Coder la fonction récursive **n\_utf8(n)** qui renvoie  $u_n$ .
4. Coder la fonction « mémoïsée » **n\_utf8\_memo(n)** qui renvoie  $u_n$ .
5. Coder la fonction **n\_utf8\_dyn(n)** qui renvoie  $u_n$  en itératif dynamique.

**Exo 9.** [algo\\_chemins.py](#)

1. On donne une grille  $3 \times 4$ .

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

On cherche à répondre à la question suivante : combien de chemins mènent du coin supérieur gauche au coin inférieur droit, en se déplaçant uniquement le long des traits horizontaux vers la droite et le long des

traits verticaux vers le bas ?

Une méthode pour y parvenir sans douleur : compléter la grille suivante qui indique sur chaque intersection le nombre de chemins qui mènent au coin inférieur droit

?

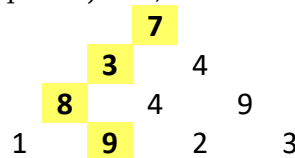
|  |  |   |   |
|--|--|---|---|
|  |  |   |   |
|  |  | 2 | 1 |
|  |  | 1 |   |

2. Ecrire une fonction **chemins(n,m)** la plus performante possible qui calcule le nombre de chemins, sur une grille  $n \times m$ .

Vérifier qu'une grille de  $10 \times 10$  admet 48520 chemins.

### Exo 10. Une pyramide de nombres

On considère une pyramide de nombres. En partant du sommet, et en se dirigeant vers le bas à chaque étape, on gagne la valeur de la case traversée. On cherche à maximiser le gain total, arrivé en bas. Sur l'image d'exemple, ce gain maximum est 27 (le chemin est indiqué en jaune).

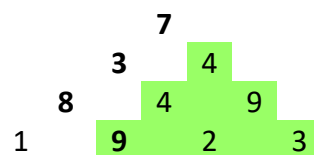
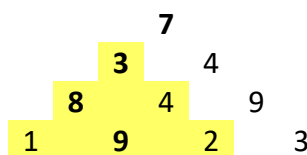


Quelle est la meilleure méthode pour trouver un tel chemin ?

1. **Choix d'une structure de données** pour représenter cette pyramide de nombres  
On choisit d'utiliser la liste suivante pour représenter la pyramide ci-dessus :

**p\_1** = [[7], [3,4], [8,4,9], [1,9,2,3]]

- a. Coder une fonction **genere\_pyramide(h)** : qui renvoie une pyramide de hauteur  $h$  d'entiers aléatoires compris entre 1 et 9 (La hauteur d'une pyramide est le nombre de ses étages. Dans l'exemple ci-dessus, la pyramide a pour hauteur 4. )
  - b. Ecrire le code Python d'une fonction **affiche\_pyramide(pyramide)** : qui affiche en mode texte la pyramide **pyramide** (exemple console à insérer ??? ).
2. **Première méthode** : examen de tous les chemins possibles
    - a. Compléter cette formulation récursive :  
**Si hauteur(pyramide) = 1 alors gain\_max(pyramide) = élément unique de la pyramide**  
**Sinon, gain\_max (pyramide) =**



- b. En déduire un code *Python* de la fonction récursive nommée **gain\_max\_naif(pyramide)** qui recherche et retourne la valeur du gain maximal en examinant tous les chemins possibles.  
tester avec la pyramide **p\_1** et vérifier que le gain maximal renvoyé est bien 27.  
Tester votre code avec la pyramide **p\_2** suivante de hauteur 10 et donner le gain maximal pour cette pyramide. :
- ```
p_2=[[8], [8, 2], [1, 8, 6], [7, 8, 2, 5], [3, 4, 3, 6, 4], [6, 9, 1, 4, 6, 5], [5, 3, 2, 2, 7, 3, 2], [6, 3, 7, 1, 1, 5, 3, 2], [9, 5, 3, 2, 8, 4, 2, 4, 7], [6, 3, 4, 9, 4, 2, 9, 1, 7, 8]]
```
- c. En utilisant un compteur du nombre d'appels de la fonction récursive, estimer la complexité algorithmique de la fonction **gain_max_naif(pyramide)** en fonction de la hauteur de la pyramide.
3. **Deuxième méthode** : de haut en bas utilisant un algorithme de programmation dynamique

On envisage de « mémoriser » les résultats calculés dans une pyramide **p_memo** de mêmes dimensions que la pyramide **p** à traverser :

```

      7
     3 4
    8 4 9
   1 9 2 3
  
```

- a. Compléter la dernière ligne de la pyramide **p_memo** à « mémoriser »

```

      7
     7
    10 11
   18 10 11 20
  18 14 11 13
  
```

- b. Coder la fonction **gain_max_memo(pyramide, p_memo=None)** correspondante. Tester avec **p_1** et **p_2**.

```
def gain_max_memo(pyramide, p_memo=None):
    if pyramide == None : return
    # au premier appel p_memo n'existe pas.
    if p_memo==None : p_memo=[[pyramide[0][0]]]
    h=len(pyramide)
    # cas de base
    # au dernier appel, on est arrivé en bas :
    # on renvoie le gain max qui alors est le max de tous les gains de la dernière ligne de
    p_memo
    pass #à compléter

    # calcul du gain : construction de p_memo en cours
    # creation de la nouvelle ligne de p_memo
    ligne =[]
    pass #à compléter

    p_memo.append(ligne)
    return gain_max_memo(pyramide, p_memo)
```

- c. Estimer (théoriquement ou en utilisant un compteur du nombre d'appels de la fonction récursive) la complexité algorithmique de la fonction **gain_max_memo(pyramide)** en fonction de la hauteur de la pyramide.
Mesurer les performances avec le module **timeit**.
- d. Compléter cette fonction afin qu'elle renvoie le chemin qui conduit au gain max.